SW Engineering CSC648/848 Fall 2023

Team 02

Milestone 4

California SafetyNet

Lukas – lheber@sfsu.edu – Team Lead, Github Master

Marshall – back end lead

Danial – front end lead

Ely, Mark, Sanjith

Date: 12/1/2023

Date submitted: 11/7/2023

Date revised: 12/1/2023

Content

1. Product summary 3

2. Usability test plan 4

3. QA test plan 6

4. Code Review 7

5. Self-check on security best practices 9

6. Adherence to original Non-functional specs 10

1. Product summary

California Safety Net

- Potential users shall be able to sign-up to the site and if needed shall then be authorized by administrators as official figures that can initiate alerts
- Registered and authorized users shall be able to initiate alerts to the general public in one “their” category, which is one of the following four: 1. Security, 2. Health, 3. Weather and 4. Fire
- Users shall be able to access a map that allows an overview of alerts for different counties
- Besides the map users shall be able to search and filter alerts to get an overview for example for planning a trip inside the state of California

California Safety Net allows its users an easy way to stay safe and informed. Registered users are alerted by county officials should it be necessary. This helps to always keep the Californian public safe. A big aid in this is the extensive map feature that allows users to get an overview of the whole state of California and its safety situation. This enables planning trips during the fire season and provides a feeling of safety to its users. Fall into the reassurance provided by California Safety Net today!

Link to California Safety Net: California SafetyNet (softwareengineering.servehttp.com)

2. Usability test plan

Test Objectives

The primary objective of this usability test is to evaluate the usability of the alert creation feature within California SafetyNet. Specifically creating, publishing, and removing alerts is to be tested. Since this is a very integral feature of California SafetyNet it is of utmost importance that the functionality can be used easily and as expected, which is why this usability test is being performed on this feature.

Test Background and Setup:

System Setup:

The test will be conducted on the latest version of the system, which includes the alert creation functionality. The system is hosted on a test server to ensure a controlled environment for testing.

Starting Point:

Users will start at the system's main page after they have logged in as an alert authorized user, this exactly represents the starting point a user using the system in the real world would have.

Intended Users:

The intended users for this test are authorized individuals (e.g. heads of one of the four departments for a county) that have gone through a brief explanation of the system's purpose and their task within.

URL of the System: California SafetyNet (softwareengineering.servehttp.com)

What is to be Measured:

For this usability test, the focus is on evaluating user satisfaction using Likert scale questions.

Usability Task description:

1. Create an alert for "your" county

How to Measure Effectiveness:

Effectiveness will be measured by the successful completion of the alert creation task. An effective completion is when the user creates an alert without errors or significant issues. Effectiveness can be determined by tracking the number of completed tasks without errors or assistance.

How to Measure Efficiency:

Efficiency will be measured by the time taken to complete the alert creation task. This metric is particularly important to understand how quickly users can perform this function.

Lickert Subjective Test:

1. It was easy to create the alert. (Answer: 1 – strongly disagree to 5 – strongly agree)
2. I feel confident creating alerts using the system. (Answer: 1 – strongly disagree to 5 – strongly agree)
3. Using the system to warn people is easier than current measures in place. (Answer: 1 – strongly disagree to 5 – strongly agree)

Space for comments:

3. QA test plan

Test Objectives:

The primary objective of this test is to evaluate the quality of the alert creation feature within California SafetyNet. Specifically creating, publishing, and removing alerts is to be tested. Since this is a very integral feature of California SafetyNet it is of utmost importance that the functionality works as expected, which is why this test is being performed on this feature.

HW and SW setup:

A modern web-browser such as Google Chrome or Microsoft Edge is needed.

URL: California SafetyNet (softwareengineering.servehttp.com)

Feature to be tested:

Creating alerts

TEST #	TEST TITLE	TEST DESCRIPTION	TEST INPUT	EXP. OUTPUT	RESULT EDGE	RESULT CHROME
1	Alert	Add an alert for the account's county	"Mass shooting happened"	"Alert submitted successfully"	Works	Works
2	Health alert	Add an alert with metric for the account's county	"Covid cases per 100K: 319"	"Alert submitted successfully"	Works	Works
3	State wide alert	Add a fire alert for the entire state by using a state department account	"Fire season, highest alert needed"	"State alert submitted successfully"	Works	Works

4. Code Review

We are using the official Google JavaScript code style which can be found here: [Google JavaScript Style Guide]

Excerpt of reviewed code:

```
+ // This sets up a POST route at '/api/alert'. When a POST request is made to this route, the following callback function will be executed.
+         app.post('/api/alert', async (req, res) => {
+             const alert = req.body.alert;
+             const timestamp = new Date(); // Get the current date and time
+
+             try {
+                 const [result] = await connection.execute(
+                     'INSERT INTO Alert (Metric_ID, Department_ID, Alert_Message, Triggered_By, Timestamp, County_ID) VALUES (?,?,?,?,?,?)',
+                     [alert.metricID, alert.departmentID, alert.alertMessage, alert.triggeredBy, timestamp, alert.countyID]
+                 );
+                 res.status(200).json(result);
+             } catch (error) {
+                 console.error('Error inserting alert:', error);
+                 res.status(500).json({ error: 'Failed to insert alert' });
+             }
+         });
+
```

Figure 1: Backend code

```
+ function triggerAlert(event) {
+     // Prevent the default form submission behavior, which typically causes a page reload.
+     event.preventDefault();
+
+     // Select the input element with the name 'alertMessage' and retrieve its current value.
+     const alert = {
+         alertMessage: document.querySelector('input[name="alertMessage"]').value,
+         triggeredBy: document.querySelector('input[name="triggeredBy"]').value,
+         countyID: document.querySelector('input[name="countyID"]').value,
+         departmentID: document.querySelector('input[name="departmentID"]').value,
+         metricID: document.querySelector('input[name="metricID"]').value
+     };
+
+     // Make an asynchronous POST request to the server's '/api/alert' endpoint.
+     fetch('/api/alert', {
+         method: 'POST', // Specify the HTTP method to use.
+         headers: { 'Content-Type': 'application/json' }, // Set the content type of the request to JSON.
+         body: JSON.stringify({ alert }) // Convert the alert message to a JSON string and set as the request body.
+     })
+         // Process the response from the fetch request.
+         .then(response => response.ok ? response.json() : Promise.reject('Failed to send alert'))
+         // If the response is successful, reset the input field for the alert message.
+         .then(() => document.querySelector('input[name="alertMessage"]').value = '')
+         // If there is an error, log it to the console.
+         .catch(console.error);
+ }
+
+ // Listen for the 'DOMContentLoaded' event, which fires when the HTML document has been completely loaded and parsed.
+ document.addEventListener('DOMContentLoaded', () => {
+     // Once the document is loaded, find the form element with the id 'alertForm'.
+     // Then add an event listener to this form that triggers the triggerAlert function when the form is submitted.
+     document.getElementById('alertForm').addEventListener('submit', triggerAlert);
+ });
```

Figure 2: Frontend code

Marshall and Mark worked together (frontend and backend) on the alert feature and submitted the changes to Lukas for review Lukas sent the following email to Marshall as code review:

Hi Marshall,

Thank you for adding the code for creating alerts! I just had a look at it and there are still some remarks I have for you to please take into consideration. As always should anything be unclear or should you otherwise require more assistance feel free to call me!

Now onto my comments:
1. Header and in-line comments are mostly missing, please add a couple of comments to help understand the code

2. Please move css styling related code to a CSS file instead of having it fill up the HTML file reducing it's readability in the process

3. I saw that instead of retrieving the counties from our database, they are hardcoded into the HTML code. Imagine a new county being added or a county changing it's name. What are the consequences using the current solution and what are the consequences using the database? Why are the counties hard coded? Furthermore, the director shouldn't have a choice of county anyway. They get one assigned and then can only create an account using that county and their department without any choice.

4. I noticed a file called data.json what is it's purpose and can we delete it? In the same way I'm wondering about the existence if three different CSS files, please merge them together (add meaningful comments to highlight the different sections).

5. Last but not least I had a hard time finding the code where the alert is inputted into the database, since in trigger_alert.js there is no INSERT INTO statement. Please either tell me where to look or add it, should that have been missed.

Enjoy the remaining part of the weekend and happy coding!

Best,
Lukas Heber

Marshall and Mark worked together to fix the remarks and comments submitted to them and responded to all the comments improving and fixing the noticed discrepancies. Only point five remained not satisfied after the initial review, prompting a follow up to include more details (county, etc.) when inserting into the database. This couldn’t be resolved in time, so Lukas had to finish it.

5. Self-check on security best practices

1. User accounts: Password is hashed with salt on the client side before being sent over to the server. Other user data is protected by general security measures

2. Data integrity: Only admin authorized accounts can push alerts. Authorization can also be revoked.

3. Server itself: Implementing the learned security practices from the lecture should ensure (excluding third party zero-day exploits, that can always exist) security

Input Validation:

To validate user input first the input length is limited to 40 characters to limit options of potential attackers. Additionally, only alphanumeric characters are allowed to further limit attack vectors. These checks are applied to all publicly available fields (search for example), but not for Officer input since this is locked behind admin authorization. The harsh limits are possible because all publicly available inputs that “make sense” are under 40 characters and can be entirely input using alphanumeric characters.

Password Encryption:

The password is salted and hashed on client side before being sent over to the backend to be stored in the database. This ensures that, no matter what unencrypted passwords can never be accessed from the server side. This is best practice since even if the server gets compromised the passwords are still safe.

6. Adherence to original Non-functional specs

Specification	Status (DONE, ON TRACK, ISSUE)
Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in M0 (some may be provided in the class, some may be chosen by the student team but all tools and servers have to be approved by class CTO).	ON TRACK
Application shall be optimized for standard desktop/laptop browsers e.g., must render correctly on the two latest versions of two major browsers	ON TRACK
Selected application functions must render well on mobile devices (this is a plus)	ON TRACK
Data shall be stored in the team's chosen database technology on the team's deployment server.	DONE
Privacy of users shall be protected, and all privacy policies will be appropriately communicated to the users.	ON TRACK
The language used shall be English.	DONE
Application shall be very easy to use and intuitive.	ON TRACK
Google maps and analytics shall be added	ON TRACK
No e-mail clients shall be allowed. You shall use webmail.	DONE
Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.	DONE
Site security: basic best practices shall be applied (as covered in the class)	ON TRACK
Modern SE processes and practices shall be used as specified in the class, including collaborative and continuous SW development	ON TRACK
The website shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Fall 2023. For Demonstration Only" at the top of the WWW page. (Important so not to confuse this with a real application).	ON TRACK